



Preprint BUW-SC 10/03

Bergische Universität Wuppertal

Fachbereich C – Mathematik und Naturwissenschaften

Mathematik

Andreas Frommer, Behnam Hashemi

## **Verified Error Bounds for Solutions of Sylvester Matrix Equations**

October 2010

<http://www.math.uni-wuppertal.de/SciComp/>

# Verified Error Bounds for Solutions of Sylvester Matrix Equations

Andreas Frommer<sup>a,\*</sup>, Behnam Hashemi<sup>b</sup>

<sup>a</sup>*Department of Mathematics and Natural Science, University of Wuppertal, 42097 Wuppertal, Germany*

<sup>b</sup>*Department of Applied Mathematics, Faculty of Mathematics and Computer Sciences, Amirkabir University of Technology, No.424, Hafez Avenue, Tehran 15914, Iran*

---

## Abstract

We develop methods for computing verified solutions of Sylvester matrix equations  $AX + XB = C$ . To this purpose we propose a variant of the Krawczyk interval operator with a factorized preconditioner so that the complexity is reduced to cubic when  $A$  and  $B$  are dense and diagonalizable. The Lyapunov equation, as a special case, is also considered.

*Key words:* Sylvester matrix equation, Lyapunov equation, interval arithmetic, Brouwer's fixed point theorem, Krawczyk's method, verified computation

*2010 MSC:* 65F05, 65G20

---

## 1. Introduction

In this paper we investigate methods for computing enclosing intervals for all entries of an  $m \times n$  solution matrix  $X$  of the Sylvester matrix equation

$$AX + XB = C, \tag{1}$$

where  $A, B$  and  $C$  are known real or complex matrices of size  $m \times m$ ,  $n \times n$ , and  $m \times n$ , respectively. Sylvester equations appear frequently in many areas of mathematics and engineering. One of the most elementary applications of (1) appears in numerical linear algebra when computing an eigenvector basis from a given Schur decomposition [1]. Sylvester equations play a vital role in control theory [2], model reduction [3, 4], the numerical solution of Riccati equations [5], image processing [6], and many other applications. We refer the reader to the article by Bhatia and Rosenthal [7] and the references therein for a history of the equation and a survey of important theoretical results. The special case  $B = A^T$  yields the Lyapunov equation which arises in several applications in control theory, e.g., in stability and robust stability, model reduction, internal balancing and determining the  $H_2$ -norm [2].

---

\*Corresponding author

*Email addresses:* `frommer@math.uni-wuppertal.de` (Andreas Frommer), `hashemi_am@aut.ac.ir`, `hoseynhashemi@gmail.com` (Behnam Hashemi)

*Preprint submitted to Elsevier*

*November 2, 2010*

The Sylvester equation (1) can be written as a system of linear equations

$$Px = c, \text{ where } P = I_n \otimes A + B^T \otimes I_m, x = \text{vec}(X) \text{ and } c = \text{vec}(C). \quad (2)$$

Herein,  $\otimes$  denotes the Kronecker product (see [8], e.g.), so  $P$  is a matrix of size  $mn \times mn$ , and  $\text{vec}$  is the operation of stacking the columns of a matrix in order to obtain one long vector. So  $\text{vec}(X)$  and  $\text{vec}(C)$  are vectors of length  $mn$ .

Let  $\sigma(A)$  and  $\sigma(B)$  denote the spectrum of  $A$  and  $B$ , respectively. We have (see [8])

$$\sigma(P) = \{\lambda_i + \mu_j \mid \lambda_i \in \sigma(A), \mu_j \in \sigma(B), i = 1, \dots, m, j = 1, \dots, n\},$$

which shows that the Sylvester equation (1) has a unique solution if and only if  $\sigma(A) \cap \sigma(-B) = \emptyset$ . In this paper we assume that this condition is always satisfied. An equivalent condition for the existence of a unique solution to (1) is that  $\begin{bmatrix} A & C \\ 0 & -B \end{bmatrix}$  be similar to  $\begin{bmatrix} A & 0 \\ 0 & -B \end{bmatrix}$  (via the similarity  $\begin{bmatrix} I & X \\ 0 & -I \end{bmatrix}$ ), thus indicating why the Sylvester equation is useful in decoupling slow and fast states in linear dynamical systems or in other decoupling problems, see e.g., [9].

Standard direct methods for solving the matrix equation (1) are the Bartels-Stewart algorithm [10] and the Hessenberg-Schur method of Golub, Nash, and Van Loan [11]. These methods are based on the Schur decomposition, by which the original equation is transformed into a form that is easy to be solved by a forward or backward substitution.

The purpose of this paper is to develop methods based on interval arithmetic which obtain guaranteed error bounds for a solution of (1). All the aforementioned classical, floating point numerical methods will always yield a result which is not an exact solution of (1) but rather an approximation to it. Given this approximation, our method aims at computing an interval enclosure for each entry of the solution matrix, i.e. a (preferably narrow) interval which is known with mathematical certainty to contain the corresponding entry of the *exact* solution of the Sylvester equation.

While there are well-established methods for computing such verified solution for general linear systems, less attention has been paid to specially structured matrix equations like (1). Seif and his collaborators [12] examined different techniques for finding the interval hull of the united solution set of an interval Sylvester equation, i.e. the smallest interval matrix  $\mathbf{X}$  containing all solutions  $X$  of Sylvester matrix equations  $AX + XB = C$ , where  $A, B, C$  are assumed to vary within interval matrices  $\mathbf{A}, \mathbf{B}, \mathbf{C}$ , resp. Most of the methods proposed in [12] have an exponential complexity in  $m$  and  $n$ . Another method proposed in [12] is a sensitivity analysis approach which is claimed to be of computational complexity  $\mathcal{O}(m^3n^2 + m^2n^3)$ . Shashikhin [13, 14] used the transformation (2) to find an interval enclosure – not necessarily the hull – for the united solution set. Rohn [15] used the same approach in the `VERMATREQN.m` code of the Versoft software. Such methods have a computational complexity of  $\mathcal{O}(m^3n^3)$ , which is very high. The approach to be presented here aims at reducing the cost to  $\mathcal{O}(m^3 + n^3)$ .

The rest of this paper is organized as follows: In section 2 we introduce some basic notation and facts from interval arithmetic. Section 3 contains our main results, including a slight extension of the theory underlying the use of Krawczyk's operator and modifications of this operator targeted for efficiency in the case of the Sylvester equation. In section 4 we present our numerical experiments and we hint towards how our method may be used to computationally prove stability when applied to Lyapunov equations. Our conclusions are summarized in section 5

## 2. Notation and preliminaries

We use the standard notation of interval analysis suggested in [16]. So  $\mathbb{IR}^n$  and  $\mathbb{IR}^{m \times n}$  denote the set of all real interval vectors and the set of all  $m \times n$  interval matrices, respectively. Similarly,  $\mathbb{IC}^n$  and  $\mathbb{IC}^{m \times n}$  denote the sets of (circular) complex interval vectors and matrices. Interval quantities will always be typeset in boldface. We assume that the reader is familiar with the basics of interval arithmetic, see [17, 18], e.g., by which the arithmetic operations in  $\mathbb{R}$  and  $\mathbb{C}$  are transported to  $\mathbb{IR}$  and  $\mathbb{IC}$ , resp. For  $\mathbb{IR}$  this is usually done via real interval arithmetic in the set theoretic sense, i.e. for any arithmetic operation  $\circ \in \{+, -, *, /\}$  one has

$$\mathbf{a} \circ \mathbf{b} = \{a \circ b : a \in \mathbf{a}, b \in \mathbf{b}\}. \quad (3)$$

The bounds of  $\mathbf{a} \circ \mathbf{b}$  can be computed from the bounds of  $\mathbf{a}$  and  $\mathbf{b}$ . For the circular complex intervals from  $\mathbb{IC}$  one usually takes *complex circular arithmetic*, where the relation (3) is relaxed to

$$\mathbf{a} \circ \mathbf{b} \supseteq \{a \circ b : a \in \mathbf{a}, b \in \mathbf{b}\}, \quad (4)$$

so that the mid point and the radius of  $\mathbf{a} \circ \mathbf{b}$  can, again, be computed from the mid points and the radii of  $\mathbf{a}$  and  $\mathbf{b}$ .

The fundamental relation (4) yields the very crucial *enclosure property* of real or complex circular interval arithmetic: If  $r(x_1, \dots, x_n)$  is an arithmetic expression in the variables  $x_1, \dots, x_n$ , then its interval arithmetic evaluation  $r(\mathbf{x}_1, \dots, \mathbf{x}_n)$  contains the range of  $r$  for  $x_1 \in \mathbf{x}_1, \dots, x_n \in \mathbf{x}_n$ .

When interval arithmetic is implemented on a computer, the parameters defining the result interval are computed in floating point arithmetic from the parameters defining the interval operands. For the enclosure property to hold for such a *machine interval arithmetic* it is therefore mandatory to use directed roundings appropriately. Software systems which provide such a reliable machine interval arithmetic are, e.g., C-XSC, see [19] and the Matlab toolbox Intlab, see [20].

For reasons of computational efficiency, Intlab uses the restriction of complex circular arithmetic to the real axis as its default arithmetic for real intervals. This results in a different multiplication and division as in standard real interval arithmetic. For the purposes of this work, we do not depend on the particular interval arithmetic in use. All we need is the enclosure property to hold, and this is true for standard real arithmetic, complex circular arithmetic as well as Intlab's default real arithmetic. For ease of notation, we from now on use  $\mathbb{K}$  to denote either of the fields  $\mathbb{R}$  or  $\mathbb{C}$  and  $\mathbb{IK}$  to denote, in the case of  $\mathbb{K} = \mathbb{R}$ , the space of real intervals equipped with standard real arithmetic or the restriction of circular arithmetic to the reals. In the case  $\mathbb{K} = \mathbb{C}$ , we assume that  $\mathbb{IK}$  is equipped with complex circular arithmetic. The interval vector and matrix spaces  $\mathbb{IK}^n$  and  $\mathbb{IK}^{n \times m}$  are to be understood accordingly.

To close this section, we recall the following facts on Kronecker products and the vec operator, see [8], e.g.

**Lemma 1.** *For any real matrices  $A, B, C$  and  $D$  with compatible sizes we have*

$$a) \ (A \otimes B)(C \otimes D) = (AC \otimes BD).$$

$$b) \ \text{vec}(ABC) = (C^T \otimes A)\text{vec}(B).$$

As was noted in [21], the above identities do not hold if we replace all matrices by interval matrices. However, due to the enclosure property, the following is still valid.

**Lemma 2.** *Let  $\mathbf{A}, \mathbf{B}, \mathbf{C}$  be interval matrices of compatible sizes. Then*

$$\{(C^T \otimes A)\text{vec}(B) : A \in \mathbf{A}, B \in \mathbf{B}, C \in \mathbf{C}\} \subseteq \begin{Bmatrix} \text{vec}((\mathbf{A}\mathbf{B})\mathbf{C}) \\ \text{vec}(\mathbf{A}(\mathbf{B}\mathbf{C})) \end{Bmatrix}.$$

### 3. A Krawczyk verification method for dense Sylvester equations

Let a linear system  $Ax = b$  with  $A \in \mathbb{K}^{N \times N}$ ,  $b \in \mathbb{K}^N$  be given together with an approximate solution  $\tilde{x} \in \mathbb{K}^N$  and an approximate inverse  $R \in \mathbb{K}^{N \times N}$  of  $A$ . In [22] Krawczyk introduced for an interval vector  $\mathbf{z} \in \mathbb{I}\mathbb{K}^N$  the operator

$$\mathbf{k}(\tilde{x}, \mathbf{z}) = -R(A\tilde{x} - b) + (I_N - RA)\mathbf{z}.$$

This operator can be used to verify that a given tentative enclosure  $\tilde{x} + \mathbf{z}$  for a solution of the linear system does indeed contain the exact solution according to the following result.

**Theorem 1.** *a) If  $\|I_N - RA\| < 1$  for some matrix norm and  $\mathbf{k}(\tilde{x}, \mathbf{z}) \subseteq \mathbf{z}$ , then  $A$  is non-singular and there exists a vector  $x^* \in \mathbf{x} := \tilde{x} + \mathbf{z}$  such that  $Ax^* = b$ .*

*b) The result under a) also holds if instead of  $\|I_N - RA\| < 1$  one requires  $\mathbf{k}(\tilde{x}, \mathbf{z}) \subset \text{int}(\mathbf{z})$ , the topological interior of  $\mathbf{z}$ .*

Part a) goes back to Krawczyk [22], part b) is due to Rump [23]. Part b) of the theorem is particularly useful, since the condition  $\mathbf{k}(\tilde{x}, \mathbf{z}) \subseteq \text{int}(\mathbf{z})$  can be directly checked computationally. The theorem holds irrespective of whether we work with complex intervals using circular arithmetic or real intervals with standard real interval arithmetic or the restriction of complex circular interval arithmetic to the reals. It is only the enclosure property that matters. To emphasize this even further, we formulate and prove the following slight generalization of part b) of Theorem 1 (see also [21, Theorem 2.3]).

**Theorem 2.** *Let  $\mathcal{Z} \subseteq \mathbb{K}^n$  be convex and compact with non-empty interior and assume that*

$$\mathcal{K}(\tilde{x}, \mathcal{Z}) := \{-R(A\tilde{x} - b) + (I_N - RA)z : z \in \mathcal{Z}\} \subset \text{int } \mathcal{Z}. \quad (5)$$

*Then the matrix  $A$  is non-singular and there exists a vector  $x^* \in \tilde{x} + \mathcal{Z}$  with  $Ax^* = b$ .*

PROOF. By Brouwer's fixed point theorem, the function  $k : z \rightarrow -R(A\tilde{x} - b) + (I_N - RA)z$  has a fixed point  $z^* \in \text{int } \mathcal{Z}$ , which implies

$$-R(A(\tilde{x} + z^*) - b) = 0. \quad (6)$$

Now, if  $R$  or  $A$  were singular, there would be a vector  $w \neq 0$  with  $RAw = 0$ , so that  $z^* + \alpha w$  would be a fixed point of  $k$  for all  $\alpha \in \mathbb{K}$ . But this line of fixed points would have a non-empty intersection with the topological boundary of  $\mathcal{Z}$ , a contradiction to (5). So  $R$  as well as  $A$  are non-singular, and (6) implies that  $x^* := \tilde{x} + z^*$  is the solution of the linear system  $Ax = b$ .

Theorem 1b) appears as a special case of this result with  $\mathcal{Z} = \mathbf{z}$ , an interval vector, and with  $\mathbf{k}(\tilde{x}, \mathbf{z})$  an interval vector containing the set  $\mathcal{K}(\tilde{x}, \mathbf{z})$  due to the enclosure property. In the real case with standard real arithmetic,  $\mathbf{k}(\tilde{x}, \mathbf{z})$  is actually the interval hull of  $\mathcal{K}(\tilde{x}, \mathbf{z})$ , see [24, Proposition 3.1.2], e.g. In our work we will consider modifications of the Krawczyk operator to obtain an interval vector containing  $\mathcal{K}(\tilde{x}, \mathbf{z})$ . These modifications will be particularly efficient, computationally, whereas they will still yield sufficiently narrow interval vectors  $\mathbf{h}$  containing  $\mathcal{K}(\tilde{x}, \mathbf{z})$ . If then  $\mathbf{h} \subseteq \text{int } \mathbf{z}$ , we also have (5), so that Theorem 2 applies.

Let us note that the crucial condition  $\mathbf{k}(\tilde{x}, \mathbf{z}) \subseteq \text{int } (\mathbf{z})$  from Theorem 1 b) or (5) from Theorem 2 is likely to hold only if  $R$  is a good approximation to the inverse of  $A$  and  $\tilde{x}$  is a good approximation to the solution of the linear system. There are then strategies for obtaining trial interval vectors  $\mathbf{z}$  around  $\tilde{x}$  for which these conditions can be expected to hold, most notably the so-called  $\epsilon$ -inflation; see [25].

**Remark 1.** If we want to compute enclosures for solutions for different right-hand side vectors  $b_1, \dots, b_q$ , we can summarize the application of the Krawczyk operator for all right-hand sides by constructing a block operator

$$\mathbf{k}(\tilde{X}, \mathbf{Z}) = -R(A\tilde{X} - B) + (I_N - RA)\mathbf{Z},$$

where  $B = [b_1 | \dots | b_q] \in \mathbb{K}^{N \times q}$ ,  $\tilde{X} \in \mathbb{K}^{N \times q}$  and  $\mathbf{Z} \in \mathbb{IK}^{N \times q}$ . This allows in particular to compute enclosures for the inverse  $A^{-1}$  of  $A$  by putting  $B = I_N$  and  $\tilde{X} = R$ .

In the case that the linear system has the form (2), i.e., the case of the Sylvester equation (1), it is very costly to evaluate the Krawczyk operator. It is in this case given as

$$\mathbf{k}(\tilde{x}, \mathbf{z}) = -R[(I_n \otimes A + B^T \otimes I_m)\tilde{x} - c] + [I_{mn} - R(I_n \otimes A + B^T \otimes I_m)]\mathbf{z}, \quad (7)$$

where  $R \in \mathbb{C}^{mn \times mn}$  is a computed (approximate) inverse of  $I_n \otimes A + B^T \otimes I_m$ . Herein,  $R$  is needed explicitly, and the product  $R(I_n \otimes A + B^T \otimes I_m)$  has to be computed explicitly, too. Note that  $R$  will usually be a dense  $mn \times mn$  matrix. Neglecting the cost to compute  $R$ , and taking the sparsity of the second factor into account, just computing the product will already require  $\mathcal{O}(m^2 n^2 (m+n))$  operations, which is prohibitively large unless  $m$  and  $n$  are small. This is the reason why we proceed to develop an alternative Krawczyk type method for the Sylvester equation now. Our method will be based on a diagonalization or a block diagonalization of  $A$  and  $B$ . As we will explain at the end of section 3.2, the standard floating point approaches of [10] and [11] which reduce  $A$  and  $B$  to triangular form via a unitary similarity transformation, is not suited for a verification method based on interval arithmetic.

### 3.1. Spectral decompositions

For simplicity, let us first assume that  $A$  and  $B$  are both diagonalizable, i.e. we have the spectral decompositions

$$A = V_A D_A W_A, \quad (8)$$

$$\text{with } V_A, W_A, D_A \in \mathbb{C}^{m \times m}, D_A = \text{diag}(\lambda_1, \dots, \lambda_m), V_A \cdot W_A = I_m,$$

$$B = V_B D_B W_B, \quad (9)$$

$$\text{with } V_B, W_B, D_B \in \mathbb{C}^{n \times n}, D_B = \text{diag}(\mu_1, \dots, \mu_n), V_B \cdot W_B = I_n.$$

So columns  $i$  of  $V_A$  and  $V_B$  are eigenvectors of  $A$  and  $B$  with eigenvalue  $\lambda_i$  and  $\mu_i$ , respectively. We then have

$$\mathbf{P} = (V_B^{-T} \otimes W_A^{-1})[I_n \otimes (W_A A W_A^{-1}) + (V_B^{-1} B V_B)^T \otimes I_m](V_B^T \otimes W_A),$$

so that defining

$$\mathbf{Q} = I_n \otimes (W_A A W_A^{-1}) + (V_B^{-1} B V_B)^T \otimes I_m, \quad (10)$$

$$y = (V_B^T \otimes W_A)x, \quad (11)$$

$$f = (V_B^T \otimes W_A)c, \quad (12)$$

we can reformulate the linear system (2) as

$$\mathbf{Q}y = f. \quad (13)$$

In a numerical computation we will obtain matrices  $V_A, W_A, D_A$  and  $V_B, W_B, D_B$  which will fulfill (8) and (9) only approximately. Let us from now on assume that  $V_A, D_A, W_B$  and  $D_B$  denote such computed approximate matrices for the spectral decompositions. The transformed system (13) is then equivalent to the original system (2) only if  $V_B^{-1}$  and  $W_A^{-1}$  are *exact* inverses. Such exact inverses cannot be obtained by a numerical computation. For our purposes this has two important implications: Firstly, we use standard interval algorithms (cf. Remark 1) to obtain interval enclosures  $\mathbf{I}_{V_B} \in \mathbb{IK}^{n \times n}$  and  $\mathbf{I}_{W_A} \in \mathbb{IK}^{m \times m}$  of  $V_B^{-1}$  and  $W_A^{-1}$ , resp., so that we have

$$\begin{aligned} W_A A W_A^{-1} &\in (W_A A) \mathbf{I}_{W_A} =: \mathbf{S}_A \text{ and} \\ V_B^{-1} B V_B &\in \mathbf{I}_{V_B} (B V_B) =: \mathbf{S}_B. \end{aligned}$$

Secondly, although  $W_A A W_A^{-1}$  as well as  $(V_B^{-1} B V_B)^T$  will not be exactly diagonal, we can expect them to be very close to  $D_A$  and  $D_B$ , respectively. Defining

$$\Delta := I_n \otimes D_A + D_B^T \otimes I_m, \quad (14)$$

we conclude that the diagonal matrix  $\Delta$  can be expected to be a good approximation for  $\mathbf{Q}$  and  $\Delta^{-1} \in \mathbb{C}^{mn \times mn}$  to be a good approximate inverse for  $\mathbf{Q}$ .

We now proceed to develop a Krawczyk type operator for  $\mathbf{Q}$  relying on  $\Delta^{-1}$  as an approximate inverse. The crucial modification to the standard Krawczyk operator is to compute an interval vector containing the set

$$\{-\Delta^{-1}(\mathbf{Q}\tilde{y} - f) + (I_{mn} - \Delta^{-1}\mathbf{Q})z : z \in \mathbf{z}\}$$

by *separately* computing enclosures for  $\{(I_n \otimes (D_A - S_A))z : z \in \mathbf{z}, S_A \in \mathbf{S}_A\}$  and  $\{((D_B - S_B)^T \otimes I_m)z : z \in \mathbf{z}, S_B \in \mathbf{S}_B\}$  using Lemma 2. Herein, all matrices  $(D_A - S_A)$  and  $(D_B - S_B)$  are close to zero. The details are given in the following proposition, where we use  $\cdot/$  to denote pointwise division in (18).

**Proposition 1.** *Let  $\tilde{X} \in \mathbb{K}^{m \times n}$  be an approximate solution of the Sylvester equation (1) and let  $\mathbf{Z} \in \mathbb{IK}^{m \times n}$  with  $\mathbf{z} = \text{vec}(\mathbf{Z})$ . Using the notation introduced before, define*

$$\mathbf{R} := W_A \cdot (A\tilde{X} + \tilde{X}B - C) \cdot V_B, \quad (15)$$

$$\mathbf{M} := (D_A - \mathbf{S}_A) \mathbf{Z}, \quad (16)$$

$$\mathbf{N} := \mathbf{Z} (D_B - \mathbf{S}_B), \quad (17)$$

$$\mathbf{U} := (-\mathbf{R} + \mathbf{M} + \mathbf{N}) ./ D \quad (18)$$

where  $D$  is the  $m \times n$  matrix with  $\text{vec}(D) = \text{diag}(\Delta)$ .

Then, with  $\tilde{y} = (V_B^T \otimes W_A)\tilde{x}$ , we have

- a)  $\{-\Delta^{-1}(Q\tilde{y} - f) + (I_{mn} - \Delta^{-1}Q)z : z \in \mathbf{z}\} \subseteq \text{vec}(\mathbf{U})$ .
- b) If  $\mathbf{U} \subset \text{int } \mathbf{Z}$ , the system  $Qy = f$  has a unique solution  $y^* \in \tilde{y} + \text{vec}(\mathbf{U})$  and  $Q$  is non-singular. Moreover, the original Sylvester equation (1) has a unique solution  $X^* \in \tilde{X} + \mathbf{V}$  where  $\mathbf{V} = \mathbf{I}_{W_A} \mathbf{U} \mathbf{I}_{V_B}$ .

PROOF. To show a) let  $\tilde{Y}$  be such that  $\text{vec}(\tilde{Y}) = \tilde{y}$  and let  $\tilde{x} = \text{vec}(\tilde{X})$ . Then

$$\begin{aligned} \text{vec}(R) &= (V_B^T \otimes W_A) \text{vec}(A\tilde{X} + \tilde{X}B - C) \\ &= (V_B^T \otimes W_A) ((I_n \otimes A)\tilde{x} + (B^T \otimes I_m)\tilde{x} - c) \\ &= (V_B^T \otimes W_A) ((I_n \otimes A) + (B^T \otimes I_m)) (V_B^{-T} \otimes W_A^{-1})(\tilde{y} - f) \\ &= Q(\tilde{y} - f). \end{aligned}$$

By Lemma 2

$$\begin{aligned} \text{vec}(\mathbf{M}) &\supseteq \{(I_n \otimes (D_A - S_A))z, z \in \mathbf{z}, S_A \in \mathbf{S}_A\} \text{ and} \\ \text{vec}(\mathbf{N}) &\supseteq \{((D_B - S_B)^T \otimes I_m)z, z \in \mathbf{z}, S_B \in \mathbf{S}_B\}. \end{aligned}$$

Since  $\Delta - Q = I_n \otimes (D_A - W_A A W_A^{-1}) + (D_B - V_B^{-1} B V_B) \otimes I_m$  and  $W_A A W_A^{-1} \in \mathbf{S}_A$ ,  $V_B^{-1} B V_B \in \mathbf{S}_B$ , we obtain

$$\{(\Delta - Q)z : z \in \mathbf{z}\} \subseteq \text{vec}(\mathbf{M}) + \text{vec}(\mathbf{N}),$$

Adding  $-\text{vec}(R)$  and multiplying with  $\Delta^{-1}$  we get a) taking into account the equivalence between multiplication with  $\Delta^{-1}$  and pointwise division with  $D$ .

The first part of b) is now an immediate consequence of Theorem 2. Transforming back, using (10)-(12), we see that  $Px = c$  has a unique solution  $x^*$  in the parallel epiped  $\tilde{x} + \{(V_B^{-T} \otimes W_A^{-1})u : u \in \mathbf{u}\}$ . With  $x^* = \text{vec}(X^*)$  this means that the original Sylvester equation (1) has a unique solution  $X^*$  in  $\tilde{X} + \{W_A^{-1} U V_B^{-1} : U \in \mathbf{U}\}$ . By the enclosure property of interval arithmetic, the set  $\{W_A^{-1} U V_B^{-1} : U \in \mathbf{U}\}$  is contained in  $\mathbf{I}_{W_A} \mathbf{U} \mathbf{I}_{V_B} = \mathbf{V}$ .

Let us note that  $\Delta$  is a computed quantity, i.e. its entries are floating point numbers. Its inverse  $\Delta^{-1}$  is usually not exactly representable in floating point. We do not use  $\Delta^{-1}$  explicitly in Proposition 1; we rather divide by the diagonal entries of  $\Delta$  in (18). In computational practice we do this using machine interval arithmetic – and thus outward rounding – so that we are sure that the computed result contains the result obtained by a multiplication with the exact inverse of  $\Delta$ .

We also note that the expressions in (15)-(18) are rich in matrix-matrix operations which is very important for reasons of efficiency. Indeed, machine interval arithmetic operations tend to be very slow compared to floating point operations. The reason is that changing the rounding mode interrupts the instruction flow on modern processors. Fortunately, this phenomenon can be avoided for matrix-matrix interval operations which can be implemented in a manner that only two changes of the rounding mode are necessary for the whole computation. This is done so for example in Rump's Intlab toolbox [20].



Algorithm 1 now describes our verification method for the Sylvester equation (1). It relies on Proposition 1 and uses standard  $\epsilon$ -inflation to get a good candidate enclosure matrix  $\mathbf{Z}$ . Herein, we use the interval hull operator  $\square(0, \mathbf{U})$ , which for a given interval matrix  $\mathbf{U}$  produces an interval matrix each entry of which is the smallest compact interval containing 0 and the respective entry of  $\mathbf{U}$ . Moreover,  $\epsilon$  in Algorithm 1 denotes the machine precision, i.e. the smallest positive floating point number  $\epsilon$  for which  $1 + \epsilon \neq 1$  when computed in floating point arithmetic ( $\epsilon = 2^{-52}$  in IEEE standard double precision).

---

**Algorithm 1** If successful this algorithm obtains an interval matrix  $\mathbf{V}$  such that  $\tilde{X} + \mathbf{V}$  contains the unique solution  $X^*$  of the Sylvester equation (1)

---

- 1: Use a floating point algorithm to get an approximate solution  $\tilde{X}$  of the Sylvester equation (1)
  - 2: Compute  $V_A, W_A, D_A, V_B, W_B$  and  $D_B$  in the spectral decompositions (8) and (9) using a floating point algorithm
  - 3: Put  $D \in \mathbb{C}^{m \times n}$  s.t.  $\text{vec}(D) = \text{diag}(D_A) + (D_B)_{jj}(1, \dots, 1)^T$
  - 4: Compute interval matrices  $\mathbf{I}_{W_A}$  and  $\mathbf{I}_{V_B}$  containing  $W_A^{-1}$  and  $V_B^{-1}$ , resp.
  - 5: {See Remark (1), implemented as `verifylss.m` in Intlab, e.g.}
  - 6: Compute  $\mathbf{R} = W_A \cdot (A\tilde{X} + \tilde{X}B - C) \cdot V_B$  using interval arithmetic
  - 7: {So rounding errors are accounted for; result  $\mathbf{R}$  contains  $R$  from (15)}
  - 8: Compute  $\mathbf{S}_A = (W_A A) \mathbf{I}_{W_A}$  and  $\mathbf{S}_B = \mathbf{I}_{V_B} (B V_B)$  using interval arithmetic everywhere
  - 9: Compute  $\mathbf{U} = -\mathbf{R} ./ D$ , put  $k = 0$  {Prepare loop}
  - 10: **repeat**
  - 11:   Put  $\mathbf{Z} = \square(0, \mathbf{U} \cdot [1 - \epsilon, 1 + \epsilon])$ , increment  $k$  { $\epsilon$ -inflation}
  - 12:   Compute  $\mathbf{M} = (D_A - \mathbf{S}_A) \mathbf{Z}$
  - 13:   Compute  $\mathbf{N} = \mathbf{Z} (D_B - \mathbf{S}_B)$
  - 14:   Compute  $\mathbf{U} = (-\mathbf{R} + \mathbf{M} + \mathbf{N}) ./ D$
  - 15:   increment  $k$
  - 16: **until** ( $\mathbf{U} \subset \text{int } \mathbf{Z}$  or  $k = 10$ )
  - 17: **if**  $\mathbf{U} \subset \text{int } \mathbf{Z}$  **then** {successful termination}
  - 18:   Output  $\mathbf{V} = \mathbf{I}_{W_A} \mathbf{U} \mathbf{I}_{V_B}$  {solution of (1) is in  $\tilde{X} + \mathbf{V}$ }
  - 19: **else**
  - 20:   Output “verification not successful”
  - 21: **end if**
- 

The following proposition stresses the gain in complexity of Algorithm 1 as compared to the standard Krawczyk operator (7).

**Proposition 2.** *Algorithm 1 has a cost of  $\mathcal{O}(m^3 + n^3)$  arithmetic operations.*

PROOF. We assume that we obtain the approximate solution  $\tilde{X}$  via the standard approach using the Schur decompositions of  $A$  and  $B$ . This has cost  $\mathcal{O}(m^3 + n^3)$ . The cost for the spectral decompositions and the computation of the enclosures  $\mathbf{I}_{W_A}$  and  $\mathbf{I}_{V_B}$  is cubic in the dimension of the respective matrices which adds up to another  $\mathcal{O}(m^3 + n^3)$ . All other matrix-matrix operations (multiplications, additions or pointwise divisions) in Algorithm 1 involve  $m \times m$ ,  $m \times n$  or  $n \times n$  matrices, so their cost is again  $\mathcal{O}(m^3 + n^3)$ . This proves the proposition since all the remaining operations in the algorithm are negligible.

### 3.2. Block diagonalization

An extension of the above approach considers the case where  $A$  and/or  $B$  are not diagonalizable or, as it will show up in computational practice, if the eigenvector matrices  $V_A$  or  $V_B$  are ill conditioned. Due to the wrapping effect, the radii of the entries of the various interval matrices to be computed in Algorithm 1 will then tend to become very large so that the condition  $\mathbf{U} \subset \text{int } \mathbf{Z}$  from part b) of Proposition 1 will not hold any more. In such a situation, we can consider *block diagonalizations* as they can be obtained using an algorithm by Bavely and Stewart [26]. We write these decompositions again as

$$A = V_A D_A W_A \quad \text{and} \quad B = V_B D_B W_B, \quad (19)$$

where, as in a spectral decomposition,  $V_A W_A = I_m$ ,  $V_B W_B = I_n$ . But now  $D_A$  and  $D_B$  are *block* diagonal with each diagonal block being triangular.<sup>1</sup> Although, in principle we can require either, upper or lower, triangular form, it will turn out advantageous to assume  $D_A$  to be *upper* triangular and  $D_B$  to be *lower* triangular from now on. If all blocks are  $1 \times 1$  we retrieve the spectral decompositions (8) and (9). The key issue is that the Bavely-Stewart algorithm allows to trade a better condition of  $V_A$  for larger diagonal blocks in  $D_A$ . An implementation of the algorithm is available as the command `bdschur` in the Matlab Control System Toolbox. It can take an upper bound for the condition number of  $V_A$  as an input and adjusts the number of diagonal blocks in  $D_A$  and their sizes accordingly.

We now formally define  $\mathbf{Q}$  exactly as in (10) and its approximation  $\Delta$  as in (14), i.e.

$$\Delta = I_n \otimes D_A + D_B^T \otimes I_m.$$

Of course,  $\Delta$  is not diagonal any more. In order to discuss its structure in more detail, let  $D_A^i \in \mathbb{C}^{m_i \times m_i}$ ,  $i = 1, \dots, M$  denote the  $M$  (upper triangular) diagonal blocks of  $D_A$ , and similarly for  $D_B$  with  $N$  (lower triangular) diagonal blocks  $D_B^i \in \mathbb{C}^{n_i \times n_i}$ ,  $i = 1, \dots, N$ . We write this as

$$D_A = \text{diag}(D_A^1, \dots, D_A^M), \quad D_B = \text{diag}(D_B^1, \dots, D_B^N).$$

Then  $\Delta$  is block diagonal with upper triangular diagonal blocks,

$$\Delta = \text{diag}(\Delta_1, \dots, \Delta_N), \quad (20)$$

$$\Delta_i = I_{n_i} \otimes D_A + (D_B^i)^T \otimes I_m \in \mathbb{K}^{mn_i \times mn_i}, i = 1, \dots, N. \quad (21)$$

Each of these blocks  $\Delta_i$  has additional internal “subblock” structure which we exemplify for the first block:

$$\Delta_1 = \begin{pmatrix} D_A + (D_B^1)_{11} \cdot I & (D_B^1)_{21} \cdot I & \cdots & (D_B^1)_{n_1 1} \cdot I \\ 0 & D_A + (D_B^1)_{22} \cdot I & \cdot & (D_B^1)_{n_1 2} \cdot I \\ 0 & 0 & \ddots & \vdots \\ 0 & 0 & \cdots & D_A + (D_B^1)_{n_1 n_1} \cdot I \end{pmatrix}.$$

---

<sup>1</sup>In order to preserve real quantities, the algorithm of Bavely and Stewart actually produces only *quasi* triangular blocks, containing real  $2 \times 2$  diagonal subblocks for any pair of conjugate eigenvalues. For simplicity, we always reduce such subblocks to a truly triangular subblock containing the complex conjugate eigenvalues via a unitary similarity transformation at the expense of having to use complex arithmetic at least from that point on.

So, if for example all blocks in  $D_A$  and  $D_B$  are  $2 \times 2$ , the sparsity structure of  $\Delta$  is given as

$$\begin{pmatrix} * & * & * & & & \\ & * & & * & & \\ & & * & * & & \\ & & & * & & \\ & & & & \ddots & \\ & & & & & \ddots \\ & & & & & & * & * & * \\ & & & & & & & * & \\ & & & & & & & & * & * \\ & & & & & & & & & * \end{pmatrix}.$$

It is easy to see that Proposition 1 also holds when we use these block diagonalizations, provided we adjust the computation of  $\mathbf{U}$  in (18) to the fact that  $\Delta$  is now (sparse) upper block triangular. Multiplying a vector with  $\Delta^{-1}$  can be done through classical back substitution. So we again do not compute  $\Delta^{-1}$  explicitly; and if we work with machine interval arithmetic and thus outward rounding, the computed result will contain  $\Delta^{-1}\text{vec}(-\mathbf{R} + \mathbf{M} + \mathbf{N})$ . Note that due to the block structure of  $\Delta$ , the back substitution process breaks into several parts, one for each block  $\Delta_i$  from (20) and that, in addition, each such block is sparse.

Algorithm 1 has to be adjusted at three places, only, to allow for the use of block diagonalizations. In line 2 we replace the computation of a spectral decomposition by the computation of the matrices in the block diagonalizations (19). In line 9 we replace the computation of  $\mathbf{U}$  by performing backward substitution on  $\Delta$  with the right hand side vector  $\text{vec}(\mathbf{R})$ . The result is a vector  $\mathbf{u}$  which we cast into a matrix  $\mathbf{U}$  with  $\mathbf{u} = \text{vec}(\mathbf{U})$  to go on. A similar modification is needed in line 14. It must be noted that the last two modifications destroy the matrix-matrix operation paradigm, so that the modified algorithm will be substantially more time consuming in lines 9 and 14 as before.

If the sizes of the blocks in  $D_A$  and  $D_B$  are bounded by a constant  $b$ , say, the back substitution process has complexity  $\mathcal{O}(nmb)$ , which ( $b$  considered to be fixed) is an order of magnitude less  $\mathcal{O}(m^3 + n^3)$ , the complexity of the remaining computations in the algorithm. So in this case we expect the modified algorithm to still be computationally efficient.

We finish this section by pointing out an important caveat to back (or forward) substitution in interval arithmetic. Consider, for example, back substitution for a  $3 \times 3$  upper triangular matrix  $A$  and a right hand side interval vector  $\mathbf{b}$ . The components of the resulting interval vector  $\mathbf{u}$ , computed via back substitution, are given as

$$\begin{aligned} \mathbf{u}_3 &= \mathbf{b}_3/a_{33}, \\ \mathbf{u}_2 &= (\mathbf{b}_2 - a_{23}\mathbf{u}_3)/a_{22} \\ &= (\mathbf{b}_2 - a_{23}(\mathbf{b}_3/a_{33}))/a_{22} \\ \mathbf{u}_1 &= (\mathbf{b}_1 - a_{12}\mathbf{u}_2 - a_{13}\mathbf{u}_3)/a_{11} \\ &= (\mathbf{b}_1 - a_{12}(\mathbf{b}_2 - a_{23}(\mathbf{b}_3/a_{33}))/a_{22} - a_{13}(\mathbf{b}_3/a_{33}))/a_{11}. \end{aligned}$$

Consider the arithmetic terms which express the components of  $\mathbf{u}$  as rational functions

of just the input data  $A$  and  $\mathbf{b}$ : The expression for  $\mathbf{u}_1$  contains  $\mathbf{b}_3$  twice. Unless we have lucky sign constellations in the components of  $A$  and  $\mathbf{b}$ , this means that the interval arithmetic evaluation yielding  $\mathbf{u}_1$  will give us an interval which is substantially larger than the set of all results of back substitutions with right hand sides  $\mathbf{b}$  varying in  $\mathbf{b}$ . The reason is that interval arithmetic cannot account for dependencies related to multiple occurrences of the same quantity. This effect gets (exponentially!) worse as the length of the back substitution process increases. We therefore can expect back substitution to yield interval vectors  $\mathbf{u}$  for which we can apply Proposition 1b) only if the length of the substitution process is short or breaks into several, independent short parts. The latter requires the block sizes in  $D_A$  and  $D_B$  to be bounded by a small constant. In particular, if we would reduce  $A$  and  $B$  to just upper triangular form, i.e. if we used Schur decompositions, the length of the backward substitution process will be prohibitively large.

#### 4. Numerical results

This section reports results of our numerical examples, focusing on computing time and on the quality of the computed enclosures. All our numerical experiments were done using Intlab; the hardware was a PC with a 2.00 GHz CPU Pentium 4 processor, 1 GB of RAM and 6 GB of virtual memory.

In all computations we obtained the approximate solution  $\tilde{X}$  via the function `lyap` of the Matlab Control System toolbox. We implemented three variants of Algorithm 1 which differ in the way they compute the interval matrix  $\mathbf{R}$  containing  $W_A(A\tilde{X} + \tilde{X}B - C)V_B$  in line 6. The radius of this interval matrix should be as small as possible, since  $\text{rad } \mathbf{R}/D$  is a lower bound for the radius of the enclosure of the error,  $\mathbf{U}$ ; see line 14 of the algorithm. Herein,  $A\tilde{X} + \tilde{X}B - C$  is a residual quantity, i.e. a small quantity computed from relatively large quantities. This means that outward rounding will yield to relatively large diameters when we compute an enclosure for  $A\tilde{X} + \tilde{X}B - C$  using Intlab's standard matrix operations. However, Intlab provides a way to compute enclosures for non-interval scalar products in simulated quadruple precision via its function `dot_.m`, see [27]. We can use this to compute enclosing intervals for all the entries in the products  $A\tilde{X}$  and  $B\tilde{X}$ . Their radii will be smaller, but the computing time will increase substantially, also because this approach deviates from the matrix-matrix operation paradigm, introducing many costly switchings of rounding modes. In our tables we mark this variant as "quadruple precision".

Recently Ozaki, Ogita and Oishi [28] proposed a new algorithm for enclosing the exact result of a matrix-matrix multiplication which is almost exclusively based on matrix operations. It can be viewed as a compromise which gives up some of the additional accuracy present in simulated quadruple precision computations in favor of a fast execution time in Intlab. We mark this as "improved precision" in our tables.

In the following tables all times are in seconds. In order to assess the quality of the enclosures computed by Algorithm 1, we look at the quantities

$$\xi_{ij} := \frac{\text{rad}(\mathbf{X}_{ij})}{\max\{|X_{ij}| : X_{ij} \in \mathbf{X}_{ij}\}}$$

for each entry of the computed enclosing interval matrix  $\mathbf{X} = \tilde{X} + \mathbf{V}$ . We can regard  $-\log_{10} \xi_{ij}$  as the *number of correct significant decimal digits*, since it roughly corresponds

to the number of digits to which the upper and the lower bounds coincide, i.e. the number of significant digits we know to be correct for every entry. In our tables we report *mrr*, the *maximum relative radius* defined as

$$mrr = \max_{i,j} \xi_{ij}$$

and its (geometric) mean *arr*, the *average relative radius* given as

$$arr = \left( \prod_{i,j} \xi_{ij} \right)^{\frac{1}{nm}}.$$

So  $-\log_{10} arr$  represents the arithmetic mean of the correct digits.

#### 4.1. Scaling

We first present a series of experiments intended to show the cubic scaling behavior of Algorithm 1. To this purpose we use the parameterized test examples from [29] which allow to construct dense Sylvester equations with arbitrary dimension and to control various condition numbers.

**Example 1.** Let  $a, b, s$  be real parameters and put

$$A = T^{-T} A_0 T^T, \quad B = T B_0 T^{-1}, \quad C = T^{-T} C_0 T^{-1},$$

where

$$\begin{aligned} A_0 &= \text{diag}(-1, -a, -a^2, \dots, -a^{n-1}), \\ B_0 &= \text{diag}(-1, -b, -b^2, \dots, -b^{n-1}), \\ C_0 &= \text{diag}(1, 2, \dots, n) \\ T &= H_2 S H_1 \text{ with} \\ H_1 &= I_n - \frac{2}{n} e e^T, \quad (e = (1, 1, \dots, 1)^T), \\ H_2 &= I_n - \frac{2}{n} f f^T, \quad (f = (-1, 1, \dots, (-1)^n)^T), \\ S &= \text{diag}(1, s, \dots, s^{n-1}). \end{aligned}$$

In our experiments we took  $a = 1.03$ ,  $b = 1.008$  and  $s = 1.001$  to make the Sylvester equation well conditioned even for large values of  $n$ . Numerical results are reported in Table 1 for dimensions  $n$  in a range from 50 to 500. In all cases, the algorithm needed just one sweep through the  $k$ -loop, i.e.  $\mathbf{U} \subset \text{int } \mathbf{Z}$  was already fulfilled for  $k = 1$ .

From the numbers in Table 1 we observe the following: The computational time indeed scales cubically with  $n$ . Using simulated quadruple precision for the residual increases the overall execution time by more than a factor of three, at least for larger dimensions. It yields very good enclosures with relative diameters equal or very close to machine precision  $\epsilon = 2^{-53} \approx 1.1 \cdot 10^{-16}$ . The improved double precision variant roughly requires 10% - 20% more computing time than with standard double precision residual computation. The quality of the enclosures obtained is significantly higher for improved

double precision residuals but it is also significantly worse than with simulated quadruple precision residuals. The timings for  $t_0$  given in parenthesis in the second column of the table represent the time that was spent in computing the floating point approximation  $\tilde{X}$  using Matlab's `lyap`. We see that, independent of the dimension, this time makes up for about 20% of the total execution time of Algorithm 1. Phrased differently, we may conclude that the additional cost for obtaining a mathematically guaranteed enclosure for the correct result via Algorithm 1 is approximately four times the cost than that for obtaining the floating point approximation.

| $n$ | Algorithm 1<br>usual double prec. |                                             | Algorithm 1<br>improved prec. [28] |                                             | Algorithm 1<br>quad. prec. |                                              |
|-----|-----------------------------------|---------------------------------------------|------------------------------------|---------------------------------------------|----------------------------|----------------------------------------------|
|     | time<br>( $t_0$ )                 | mrr<br>arr                                  | time                               | mrr<br>arr                                  | time                       | mrr<br>arr                                   |
| 50  | 0.1<br>(0.0)                      | $6.0 \cdot 10^{-8}$<br>$1.8 \cdot 10^{-11}$ | 0.1                                | $3.6 \cdot 10^{-9}$<br>$1.2 \cdot 10^{-12}$ | 0.2                        | $2.2 \cdot 10^{-16}$<br>$1.6 \cdot 10^{-16}$ |
| 100 | 0.3<br>(0.0)                      | $1.1 \cdot 10^{-6}$<br>$1.9 \cdot 10^{-10}$ | 0.4                                | $1.5 \cdot 10^{-8}$<br>$3.4 \cdot 10^{-12}$ | 0.6                        | $2.2 \cdot 10^{-16}$<br>$1.6 \cdot 10^{-16}$ |
| 200 | 1.8<br>(0.3)                      | $6.3 \cdot 10^{-6}$<br>$2.7 \cdot 10^{-9}$  | 2.4                                | $1.9 \cdot 10^{-8}$<br>$1.3 \cdot 10^{-11}$ | 3.9                        | $2.2 \cdot 10^{-16}$<br>$1.6 \cdot 10^{-16}$ |
| 300 | 6.2<br>(1.4)                      | $5.9 \cdot 10^{-3}$<br>$3.0 \cdot 10^{-8}$  | 8.0                                | $1.3 \cdot 10^{-5}$<br>$4.9 \cdot 10^{-11}$ | 20.7                       | $2.2 \cdot 10^{-16}$<br>$1.6 \cdot 10^{-16}$ |
| 400 | 15.4<br>(3.9)                     | $1.3 \cdot 10^{-1}$<br>$3.1 \cdot 10^{-7}$  | 19.0                               | $8.1 \cdot 10^{-5}$<br>$1.3 \cdot 10^{-10}$ | 52.2                       | $7.3 \cdot 10^{-16}$<br>$1.6 \cdot 10^{-16}$ |
| 500 | 30.2<br>(7.8)                     | $2.4 \cdot 10^{-1}$<br>$4.5 \cdot 10^{-6}$  | 37.0                               | $8.3 \cdot 10^{-5}$<br>$8.7 \cdot 10^{-10}$ | 104.7                      | $1.9 \cdot 10^{-15}$<br>$1.6 \cdot 10^{-16}$ |

Table 1: Results for Example 1.  $t_0$  stands for the time spent in the Matlab command `lyap`.

For this example we also tried Versoft [15] which uses the standard Krawczyk operator on the large system (2) and thus has complexity  $\mathcal{O}(m^3n^3)$ . Indeed, already for  $n = 50$  its computing time was more than 116s and thus about one thousand times higher than with Algorithm 1. The accuracy obtained was slightly better than Algorithm 1 with standard double precision residuals and slightly worse than with improved double precision residuals. Because of the tremendous computational cost we did not produce results for the larger values of  $n$ .

#### 4.2. Results for the CTDSX benchmark collection

We now turn to a series of experiments with data from the CTDSX benchmark collection. This is an established collection of benchmark examples for state-space realizations of continuous-time dynamical systems including various real applications like reactor or air plane control; see [30]. We took those examples which have the form

$$\begin{aligned} x'(t) &= Dx(t) + Eu(t) \\ y(t) &= Fx(t) + Gu(t), \end{aligned}$$

with  $D \in \mathbb{R}^{n \times n}, E \in \mathbb{R}^{n \times m}$ . From  $D$  and  $E$  we build the matrices  $A = D$ ,  $B = A^T$ ,  $C = -EE^T$  to form the Sylvester equation

$$AX + XA^T = C,$$

the solution of which represents the (infinite) reachability Grammian of the dynamical system; see [3], e.g. Note that we are in presence of a Lyapunov equation, here, i.e. a Sylvester equation with  $B = A^T$  and  $C$  symmetric. So the solution  $X$  is a symmetric matrix, too.

| Example<br>Number<br>in [30], $n$<br>sep | Versoft[15]<br>and [12–14] |                                              | Algorithm 1<br>improved prec. [28] |                                              | Algorithm 1<br>quad. prec. |                                              |
|------------------------------------------|----------------------------|----------------------------------------------|------------------------------------|----------------------------------------------|----------------------------|----------------------------------------------|
|                                          | time                       | mrr<br>arr                                   | time                               | mrr<br>arr                                   | time                       | mrr<br>arr                                   |
| 1.3, 4<br>$1.9 \cdot 10^{-2}$            | 0.0                        | 1.0<br>$1.2 \cdot 10^{-13}$                  | 0.0                                | 1.0<br>$1.4 \cdot 10^{-12}$                  | 0.0                        | 1.0<br>$1.3 \cdot 10^{-14}$                  |
| 1.10, 8<br>$2.2 \cdot 10^{-2}$           | 0.0                        | 1.0<br>$8.7 \cdot 10^{-14}$                  | 0.1                                | 1.0<br>$1.8 \cdot 10^{-13}$                  | 0.1                        | 1.0<br>$4.6 \cdot 10^{-15}$                  |
| 1.8, 9<br>$1.8 \cdot 10^{-13}$           | 0.0                        | $8.2 \cdot 10^{-12}$<br>$7.0 \cdot 10^{-14}$ | 0.0                                | $2.7 \cdot 10^{-13}$<br>$1.4 \cdot 10^{-14}$ | 0.0                        | $2.2 \cdot 10^{-16}$<br>$1.6 \cdot 10^{-16}$ |
| 1.6, 30<br>$6.1 \cdot 10^{-6}$           | 4.7                        | 1.0<br>$9.8 \cdot 10^{-14}$                  | 0.1                                | 1.0<br>$4.1 \cdot 10^{-13}$                  | 0.1                        | 1.0<br>$1.3 \cdot 10^{-15}$                  |
| 3.2, 40<br>$1.2 \cdot 10^{-1}$           | 17.6                       | $4.1 \cdot 10^{-13}$<br>$1.7 \cdot 10^{-13}$ | 0.1                                | $2.1 \cdot 10^{-12}$<br>$1.0 \cdot 10^{-13}$ | 0.1                        | $2.2 \cdot 10^{-16}$<br>$1.5 \cdot 10^{-16}$ |
| 1.9, 55<br>$7.6 \cdot 10^{-9}$           | 660.3                      | 1.0<br>$1.6 \cdot 10^{-11}$                  | 0.3                                | 1.0<br>$7.1 \cdot 10^{-11}$                  | 0.3                        | 1.0<br>$1.7 \cdot 10^{-13}$                  |
| 3.4, 421<br>-                            | -                          | -<br>-                                       | 48.6                               | 1.0<br>$1.9 \cdot 10^{-9}$                   | 81.0                       | $6.4 \cdot 10^{-1}$<br>$1.7 \cdot 10^{-16}$  |

Table 2: Results for Sylvester equations from CTDSX benchmark collection [30].

Table 2 reports our numerical results. The first column gives a reference to the example in [30] and the dimensions  $m$  and  $n$ . It also contains the value  $sep$  defined as the distance between  $\sigma(A)$  and  $\sigma(B)$ , i.e. the smallest eigenvalue of  $P$  from (2). So  $sep$  is related to the condition of the Sylvester equation. We do not reproduce the results for our Algorithm 1 with standard double precision residual computation, since it is only slightly faster than the version with improved accuracy for the residual but gives notably less narrow enclosures for the solution. Table 2 confirms our finding from the first numerical experiments: Simulated quadruple precision residual computation substantially improves the enclosures obtained; Versoft takes too much time on larger systems. Note also that  $mrr = 1$  indicates that the enclosure  $\mathbf{X}$  of the solution has entries which contain 0 at one of its boundaries. As before, Algorithm 1 needed one sweep through the  $k$ -loop only in all cases. We also note that a small value of  $sep$  does not necessarily mean that the enclosures we obtain will become wider. Indeed, as we will see in the next experiments, it is the condition of the eigenvector matrices of  $A$  and  $B$  which plays a vital role in this context.

#### 4.3. Results with block diagonalization

We now turn to the CTLEX collection of continuous time Lyapunov equations; see [31]. CTLEX directly gives the matrices  $A$  and  $C$  in the equation

$$AX + XA^T = C.$$

The examples from [31] are “academic” ones in the sense that one can adjust parameters like the dimension and others to make the solution of the equation more or less difficult. All matrices  $A$  are square matrices, so we report only the values of  $n$  in the first column of Table 3 together with our choice for the respective parameters. When successful, Algorithm 1 terminated after the first sweep of the  $k$ -loop. NaN for Versoft indicates that it failed to obtain an enclosure.

Example 4.1 from [31] takes the same matrix  $A$  as Example 1 above. Example 4.2 produces a matrix  $A$  which, in theory, has just one single Jordan block. Actually, since the matrix  $A$  is numerically computed as a product of several matrices, the input  $A$  to our algorithm will just be very close to a matrix with such a large Jordan block. This implies that the matrix  $V_A$  computed in the spectral decomposition will be very ill conditioned.

| Ex. no., $n$<br>in [31]<br>parameters<br>$sep$                      | Versoft[15]<br>and [12–14] |                                             | Algorithm 1<br>impr. prec. [28] |                                             | Algorithm 1<br>quad. prec. |                                              |
|---------------------------------------------------------------------|----------------------------|---------------------------------------------|---------------------------------|---------------------------------------------|----------------------------|----------------------------------------------|
|                                                                     | time                       | mrr<br>arr                                  | time                            | mrr<br>arr                                  | time                       | mrr<br>arr                                   |
| 4.1, $n = 10$<br>$r = 1.2, s = 3.0$<br>$4.0 \cdot 10^{-6}$          | 0.0                        | $3.1 \cdot 10^{-7}$<br>$4.6 \cdot 10^{-8}$  | 0.0                             | $8.8 \cdot 10^{-8}$<br>$2.6 \cdot 10^{-8}$  | 0.1                        | $2.2 \cdot 10^{-16}$<br>$1.5 \cdot 10^{-16}$ |
| 4.1, $n = 15$<br>$r = 2.3, s = 2.5$<br>$1.1 \cdot 10^{-8}$          | 0.1                        | NaN<br>NaN                                  | 0.0                             | $5.9 \cdot 10^{-4}$<br>$1.9 \cdot 10^{-4}$  | 0.1                        | $1.8 \cdot 10^{-13}$<br>$5.9 \cdot 10^{-14}$ |
| 4.1, $n = 50$<br>$r = 1.3, s = 1.1$<br>1.6                          | 141.4                      | $3.0 \cdot 10^{-6}$<br>$1.2 \cdot 10^{-9}$  | 0.1                             | $1.0 \cdot 10^{-8}$<br>$2.3 \cdot 10^{-11}$ | 0.2                        | $2.2 \cdot 10^{-16}$<br>$1.6 \cdot 10^{-16}$ |
| 4.2, $n = 31$<br>$s = 1.2, \lambda = -1.1$<br>$7.7 \cdot 10^{-5}$   | 5.4                        | $1.4 \cdot 10^{-9}$<br>$2.2 \cdot 10^{-12}$ | 0.1                             | 1.0<br>1.0                                  | 0.3                        | 1.0<br>1.0                                   |
| 4.2, $n = 25$<br>$s = 1.6, \lambda = -1.1$<br>$1.3 \cdot 10^{-9}$   | 1.9                        | $5.1 \cdot 10^{-4}$<br>$4.1 \cdot 10^{-5}$  | 0.1                             | 1.0<br>1.0                                  | 0.1                        | 1.0<br>1.0                                   |
| 4.2, $n = 20$<br>$s = 2.09, \lambda = -1.1$<br>$4.7 \cdot 10^{-12}$ | 1.1                        | NaN<br>NaN                                  | 0.1                             | 1.0<br>1.0                                  | 0.1                        | $9.3 \cdot 10^{-1}$<br>$1.7 \cdot 10^{-1}$   |

Table 3: Results for Lyapunov equations from [31].

In the fourth row of Table 3 we have a Lyapunov equation with  $sep = 7.7 \cdot 10^{-5}$ . Even though Algorithm 1 can be successful for examples with much smaller values of  $sep$ , here it failed. The reason is that  $V_A$  is very ill-conditioned. Indeed, Matlab’s command `condeig` returns a condition number of  $10^{+14}$ . As a consequence, the interval matrix  $\mathbf{S}_A$  in Algorithm 1 has an approximate radius of  $10^{+10}$ , the reason why we fail to satisfy the



condition  $\mathbf{U} \subset \mathbf{Z}$ . The situation is similar for rows five and six, where the approximate eigenvalue condition numbers are  $10^{+11}$  and  $10^{+10}$ , respectively.

| $n$ | block diag.<br>usual double prec. |                                            | block diag.<br>improv. prec. [28] |                                            | block diag.<br>quad. prec. |                                              |
|-----|-----------------------------------|--------------------------------------------|-----------------------------------|--------------------------------------------|----------------------------|----------------------------------------------|
|     | time                              | mrr<br>arr                                 | time                              | mrr<br>arr                                 | time                       | mrr<br>arr                                   |
| 31  | 19.4                              | $8.2 \cdot 10^{-6}$<br>$3.4 \cdot 10^{-8}$ | 19.5                              | $1.3 \cdot 10^{-6}$<br>$5.3 \cdot 10^{-9}$ | 19.6                       | $2.2 \cdot 10^{-16}$<br>$1.5 \cdot 10^{-16}$ |
| 25  | 10.5                              | 1.0<br>$9.1 \cdot 10^{-1}$                 | 10.5                              | $1.8 \cdot 10^{-1}$<br>$9.1 \cdot 10^{-3}$ | 10.6                       | $1.3 \cdot 10^{-13}$<br>$5.6 \cdot 10^{-15}$ |
| 20  | 5.7                               | 1.0<br>1.0                                 | 5.7                               | 1.0<br>1.0                                 | 5.7                        | $3.9 \cdot 10^{-7}$<br>$6.6 \cdot 10^{-9}$   |

Table 4: Results for the last three examples of Table 3 with block diagonalization.

Table 4 reports the results using the modification of Algorithm 1 based on block-diagonalization for the last three ill-conditioned examples of Table 3. Here we asked for a condition number in  $V_A$  of  $1/\sqrt{\epsilon} \approx 6.7 \times 10^7$ , the default value in `bdschur`. This resulted in  $D_A$  consisting of just one diagonal block. The variant with simulated quadruple precision evaluation of the residual is now capable to get enclosures for the solution of all examples. When successful, the condition  $\mathbf{U} \subset \text{int } \mathbf{Z}$  was verified in the first sweep through the loop over  $k$ . More than 90% of the total computing time reported in Table 4 was spent in the two calls to the backward substitution routine needed in lines 9 and 14 of the (modified) Algorithm 1. Backward substitution is particularly costly in computing time not because of its arithmetic operations but because of the many switching of rounding modes when done in interval arithmetic.

As a last example in this section we took the Sylvester equation with  $m = n = 50$  and  $A$  and  $B$  to be the matrices `gearmat` and `gcdmat` from Matlab's gallery. So  $B$  is real symmetric and thus unitarily diagonalizable, whereas  $A$  has one Jordan block of size 2.  $C$  was taken as a random matrix. In the block diagonalization for  $B$  we asked again for a condition number of  $1/\sqrt{\epsilon}$  for  $V_B$ , and `bdschur` produced a matrix  $D_B$  which is diagonal except for one  $2 \times 2$  block. With improved precision in the block diagonalization version of Algorithm 1 we needed a total time of 4.9s and obtained  $mrr = 2.2 \cdot 10^{-10}$  and  $arr = 2.6 \cdot 10^{-13}$ , whereas the simulated quadruple precision version needed 3.2s and got  $mrr = 2.2 \cdot 10^{-16}$  and  $arr = 1.6 \cdot 10^{-16}$ . In both variants the back substitution took more than 90% of the overall execution time. The fact that simulated quadruple precision performed faster is an anomaly which we attribute to memory allocation and memory traffic effects.

## 5. Conclusions

In this paper we presented a method to obtain guaranteed interval enclosures for the solutions of Sylvester and Lyapunov equations. The method starts from a traditionally computed approximate solution  $\tilde{X}$  of the equation and then computes, with mathematical rigor, an enclosing interval matrix  $\mathbf{U}$  for the error of  $\tilde{X}$ . The method is computationally efficient in theory, since its complexity is cubic, and it is efficient in practice since it relies

mainly on matrix-matrix operations which are favorable to the fast execution of interval arithmetic. Our Intlab implementations show that the computation of the error bound  $\mathbf{U}$  is roughly just four times as costly as that for computing the approximate solution  $\tilde{X}$ , and that the quality of the enclosure, i.e. the radii of the enclosing intervals, is usually quite satisfactory. If the diagonalization of  $A$  or  $B$  results in very ill-conditioned eigenvector matrices, our method may fail in the sense that it will tell us that it did not succeed in computing an enclosure for the error. A remedy to such situations is the use of block diagonalizations at the expense of a substantial increase in computing time in Intlab.

In stability theory one often is confronted with the problem to prove that the solution to a given Lyapunov equation is (positive or negative) semidefinite. Our methods contribute an element for performing such a proof computationally: One first computes an enclosure  $\mathbf{X}$  for the solution of the Lyapunov equation. Then one applies the rigorous approach from [32], which relies on perturbation theory and interval arithmetic, implemented as Intlab's function `isspd`, to  $\mathbf{X}$ . If successful, one will have proven the definiteness of all symmetric matrices contained in  $\mathbf{X}$ , so in particular of the solution of the Lyapunov equation.

*Acknowledgement.* The authors would like to thank Siegfried M. Rump and Bernd Tibken for a stimulating discussion when the first author presented part of this work at the Dagstuhl seminar 09471, "Computer-assisted proofs - tools, methods and applications".

## References

- [1] G. Golub, C. Van Loan, Matrix Computations, Johns Hopkins University Press, Baltimore, Maryland, 1996.
- [2] B. Datta, Numerical Methods for Linear Control Systems, Elsevier Academic Press, Amsterdam, 2004.
- [3] A. Antoulas, Approximation of Large-Scale Dynamical Systems, Advances in Design and Control, SIAM, Philadelphia, 2005.
- [4] D. Sorensen, A. Antoulas, The Sylvester equation and approximate balanced reduction, Linear Algebra Appl. 351/352 (2002) 671–700.
- [5] C. Choi, A. Laub, Efficient matrix-valued algorithm for solving stiff Riccati differential equations, IEEE Trans. Automat. Control 35 (1990) 770–776.
- [6] D. Calvetti, L. Reichel, Application of ADI iterative methods to the restoration of noisy images, SIAM J. Matrix Anal. Appl. 17 (1996) 165–186.
- [7] R. Bhatia, P. Rosenthal, How and why to solve the operator equation  $AX - XB = Y$ , Bull. London Math. Soc. 29 (1997) 1–21.
- [8] R. A. Horn, C. R. Johnson, Topics in Matrix Analysis, Cambridge University Press, Cambridge, 1994.
- [9] M. Epton, Methods for the solution of  $AXD - BXC = E$  and its application in the numerical solution of implicit ordinary differential equations, BIT 20 (1980) 341–345.
- [10] R. Bartels, G. Stewart, Solution of the matrix equation  $AX + XB = C$ , Comm. ACM 15 (1972) 820–826.
- [11] G. Golub, S. Nash, C. Van Loan, Hessenberg-Schur method for the problem  $AX + XB = C$ , IEEE Trans. Automat. Control AC-24 (1979) 909–913.
- [12] N. Seif, S. Hussein, A. Deif, The interval Sylvester matrix equation, Computing 52 (1994) 233–244.
- [13] V. Shashikhin, Robust assignment of poles in large-scale interval systems, Automation and Remote Control 63 (2002) 200–208.
- [14] V. Shashikhin, Robust stabilization of linear interval systems, J. Appl. Maths Mechs. 66 (2002) 393–400.
- [15] J. Rohn, VERSOFT: Verification software in MATLAB/INTLAB, <http://uivtx.cs.cas.cz/~rohn/matlab>.

- [16] R. B. Kearfott, M. Nakao, A. Neumaier, S. Rump, S. Shary, P. van Hentenryck, Standardized notation in interval analysis, <http://www.mat.univie.ac.at/~neum/ms/notation.pdf> (2005).
- [17] G. Alefeld, J. Herzberger, Introduction to Interval Computations, Computer Science and Applied Mathematics, Academic Press, New York, 1983.
- [18] R. E. Moore, R. B. Kearfott, M. J. Cloud, Introduction to Interval Analysis, SIAM, Philadelphia, 2009.
- [19] R. Klatte, U. W. Kulisch, A. Wiethoff, C. Lawo, M. Rauch, C-XSC. A C++ Class Library for Extended Scientific Computing, Springer-Verlag, Berlin, 1993.
- [20] S. M. Rump, INTLAB – INTerval LABoratory, in: T. Csendes (Ed.), Developments in Reliable Computing, Kluwer Academic Publishers, Dordrecht, 1999, pp. 77–104.
- [21] A. Frommer, B. Hashemi, Verified computation of square roots of a matrix, SIAM J. Matrix Anal. Appl. 31 (2010) 1279–1302.
- [22] R. Krawczyk, Newton-Algorithmen zur Bestimmung von Nullstellen mit Fehlerschranken, Computing 4 (1969) 187–201.
- [23] S. M. Rump, Solving algebraic problems with high accuracy, in: W. Miranker, E. Kaucher (Eds.), A New Approach to Scientific Computation, Vol. 7 of Comput. Sci. Appl. Math., Academic Press, New York, 1983, pp. 51–120.
- [24] A. Neumaier, Interval Methods for Systems of Equations, no. 37 in Encyclopedia of Mathematics and its Applications, Cambridge University Press, Cambridge, 1990.
- [25] S. M. Rump, A note on epsilon-inflation, Reliab. Comput. 4 (1998) 371–375.
- [26] A. Bavelly, G. Stewart, An algorithm for computing reducing subspaces by block diagonalization, SIAM J. Numer. Anal. 16 (1979) 359–367.
- [27] T. Ogita, S. M. Rump, S. Oishi, Accurate sum and dot product, SIAM J. Sci. Comput. 26 (2005) 1955–1988.
- [28] K. Ozaki, T. Ogita, S. Oishi, Tight and efficient enclosure of matrix multiplication by using optimized BLAS, Numerical Linear Algebra with Applications, (Early View) xx (2010) xxx–xxx.
- [29] P. Benner, V. Sima, M. Slowik, Evaluation of the linear matrix equation solvers in SLICOT, Journal of Numerical Analysis, Industrial and Applied Mathematics 2 (2007) 11–34.
- [30] D. Kressner, V. Mehrmann, T. Penzl, CTDSX - A collection of benchmark examples for state-space realizations of continuous-time dynamical systems, Tech. Rep. SLICOT Working Note 1998-9., <http://www.slicot.org/REPORTS/SLWN1998-9.ps.gz> (1998).
- [31] D. Kressner, V. Mehrmann, T. Penzl, CTLEX - A collection of benchmark examples for continuous-time Lyapunov equations, Tech. Rep. SLICOT Working Note 1999-6., <http://www.slicot.org/REPORTS/SLWN1999-6.ps.gz> (1999).
- [32] S. M. Rump, Verification of positive definiteness, BIT Numerical Mathematics 46 (2006) 433–452.